Noesis architecture

Nov 12, 2026 · @coconut

Overview

Noesis is a local study assistant: an Angular app calls a FastAPI backend, which stores PDFs on disk and in Postgres with pgvector, embeds text locally with Qwen3, and asks Groq for cited answers and quizzes.

- Frontend: `Frontend/Noesis/src/app` (Angular 22, standalone components, signals).
- Backend: `Backend/src/backend` (FastAPI, SQLAlchemy 2, Alembic). Paths below are relative to these two folders unless written in full.
- Database: Postgres 17 + pgvector in Docker (`Backend/docker-compose.yml`), schema in `Backend/alembic/versions/0001_initial_schema.py` and `0002_quizzes.py` .
- Auth is off: every request acts as one local user, created in `api/dependencies.py:27` (`get_current_user`).

Line numbers point at where each class or function starts, as of this date.

Components

Each layer only calls the one below it: pages call API services, routes call services, services call providers and the database.

```
flowchart TB
  subgraph FE["Angular frontend  (Frontend/Noesis/src/app)"]
    Pages["Pages<br/>features/*"] --> ApiSvc["API
services<br/>core/services/*"]
    ApiSvc --> Interceptors["authInterceptor, errorInterceptor<br/>core/auth,
core/http"]
    Pages --> Viewer["PdfViewer<br/>shared/ui/pdf-viewer.ts"]
  end
  Interceptors -->|"HTTP /api/v1 (dev proxy)"| Routes
  subgraph BE["FastAPI backend  (Backend/src/backend)"]
    Routes["Routes<br/>api/routes/*"] -->
Deps["Dependencies<br/>api/dependencies.py"]
    Routes --> Services["Services<br/>services/*"]
    Services --> RAG["RAG pipeline + prompts<br/>rag/pipeline.py,
rag/prompts"]
    Services --> Providers["Providers<br/>rag/providers/*"]
  end
  Providers --> Qwen["Qwen3-Embedding-0.6B<br/>local GPU/CPU"]
  Providers --> Groq["Groq API<br/>llama-3.3-70b"]
  Services --> DB[("Postgres + pgvector")]
  Services --> Disk[("STORAGE_DIR<br/>copied PDFs")]
```

Layer	Folder	Job
Pages	`features/`	Screen state (signals), user actions
API services	`core/services/`	One `HttpClient` call per endpoint, no logic
Routes	`api/routes/`	Validate input, call a service, map errors to HTTP codes
Services	`services/`	Business logic and database work
RAG	`rag/`	Prompts, answer normalising, provider interfaces and implementations
Models	`models/`	SQLAlchemy tables
Schemas	`schemas/`	Pydantic request and response shapes

Backend classes

The backend has three groups of classes: database models that hold the data, service modules that hold the logic, and provider classes behind interfaces so the embedder and LLM can be swapped.

Data model

A user owns documents, conversations and quizzes; a document is split into pages and chunks; a message's sources are snapshots that survive document deletion.

```
classDiagram
  class User {
    id: UUID
    email: str
    default_retrieval_scope: RetrievalScope
  }
  class Document {
    id: UUID
    filename: str
    sha256: str
    status: DocumentStatus
    page_count: int
    error: str
  }
  class DocumentPage {
    page_number: int
    char_count: int
    needs_ocr: bool
    extraction_method: PageExtraction
  }
  class DocumentChunk {
    page_number: int
    kind: ChunkKind
    text: str
    embedding: vector(1024)
  }
  class Conversation {
    title: str
    mode: LearningMode
    retrieval_scope: RetrievalScope
  }
  class Message {
```

```
    role: MessageRole
    content: str
    answer_status: AnswerStatus
    supplementary: str
  }
  class MessageSource {
    position: int
    filename: str
    page_number: int
    quote: str
    document_id: UUID nullable
  }
  class Quiz {
    title: str
    topic: str
  }
  class QuizQuestion {
    prompt: str
    options: list~str~
    correct_index: int
    explanation: str
  }
  class QuizAttempt {
    answers: list~int~
    score: int
  }
  User "1" --> "*" Document
  User "1" --> "*" Conversation
  User "1" --> "*" Quiz
  Document "1" *-- "*" DocumentPage
  Document "1" *-- "*" DocumentChunk
  Conversation "*" -- "*" Document : conversation_documents
  Conversation "1" *-- "*" Message
  Message "1" *-- "*" MessageSource
  MessageSource ..> DocumentChunk : snapshot of
  Quiz "1" *-- "*" QuizQuestion
  Quiz "1" *-- "*" QuizAttempt
```

Class	File	Holds
`User`	`models/user.py:8`	The local user and the default scope setting

Class	File	Holds
`Document`	`models/document.py:23`	One uploaded PDF and its status
`DocumentPage`	`models/document.py:55`	Per-page text stats, `needs_ocr` flag
`DocumentChunk`	`models/document_chunk.py:25`	A chunk of one page plus its 1024-dim vector (HNSW index)
`Conversation`	`models/conversation.py:26`	A chat, its mode, scope and attached PDFs
`Message`	`models/message.py:25`	A user question or assistant answer
`MessageSource`	`models/message.py:72`	Citation snapshot: filename, page, quote
`Quiz`, `QuizQuestion`, `QuizAttempt`	`models/quiz.py:11`, `:38`, `:61`	Quizzes, their questions and graded attempts
Enums	`models/enums.py`	`DocumentStatus`, `LearningMode`, `RetrievalScope`, `AnswerStatus`, …
`Base`, `UUIDPk`, `Timestamps`	`database/base.py:20`, `:24`, `:28`	Shared id and timestamp columns

Providers

Services depend on the two Protocols, not on Qwen3 or Groq directly; `rag/factory.py` picks the implementation from settings.

```
classDiagram
  class EmbeddingProvider {
    <<Protocol>>
    dimension: int
    embed_documents(texts) list~vector~
    embed_query(text) vector
  }
```

```
class LLMProvider {
  <<Protocol>>
  complete(messages, json_mode) LLMResult
}
class OCRProvider {
  <<Protocol>>
  extract_page_text(page) str
}
class Qwen3EmbeddingProvider {
  -_model: SentenceTransformer
  -_load()
  -_encode(texts, prompt)
}
class GroqLLMProvider {
  -_client: OpenAI
}
class FakeEmbedder
class FakeLLM
EmbeddingProvider <|.. Qwen3EmbeddingProvider
EmbeddingProvider <|.. FakeEmbedder
LLMProvider <|.. GroqLLMProvider
LLMProvider <|.. FakeLLM
class Factory {
  build_embedder(settings)
  build_llm(settings)
  get_embedder()
  get_llm()
}
Factory ..> Qwen3EmbeddingProvider : creates
Factory ..> GroqLLMProvider : creates
```

Class	File
`LLMProvider`, `EmbeddingProvider`, `OCRProvider`, `ChatMessage`, `LLMResult`	`rag/interfaces.py:22`, `:30`, `:39`, `:10`, `:16`
`Qwen3EmbeddingProvider`	`rag/providers/embeddings/qwen3.py:10`
`GroqLLMProvider`	`rag/providers/llm/groq.py:10`
Factory functions	`rag/factory.py:17` (`build_llm`), `:33` (`build_embedder`)

Class	File
`FakeEmbedder`, `FakeLLM` (tests only)	`Backend/tests/fakes.py`

Services

Services are plain modules of functions, called by routes with a database session.

```
classDiagram
  class document_service {
    upload_document()
    delete_document()
    mark_missing_files()
  }
  class ingestion {
    ingest_document()
    pending_document_ids()
  }
  class pdf_extraction {
    extract_pdf()
    looks_like_pdf()
  }
  class chunking {
    chunk_pages()
    chunk_page()
  }
  class retrieval {
    retrieve()
    build_retrieval_query()
  }
  class answer_service {
    ask() AnswerResult
  }
  class pipeline {
    build_messages()
    normalize() NormalizedAnswer
  }
  class quiz_service {
    generate_quiz()
    submit_attempt()
  }
  class conversation_service {
```

```
    create_conversation()
    recent_history()
  }
  class FileStorage {
    save()
    path()
    delete()
  }
  document_service --> FileStorage
  document_service --> ingestion : schedules
  ingestion --> pdf_extraction
  ingestion --> chunking
  ingestion --> EmbeddingProvider
  answer_service --> conversation_service
  answer_service --> retrieval
  answer_service --> pipeline
  answer_service --> LLMProvider
  retrieval --> EmbeddingProvider
  quiz_service --> retrieval
  quiz_service --> LLMProvider
```

Function / class	File	What it does
`upload_document`	`services/document_service.py:37`	Checks size and PDF header, sha256 dedupe, saves the file, inserts `Document(pending)`
`delete_document`	`services/document_service.py:125`	Deletes file, row, chunks; citations keep their snapshot
`mark_missing_files`	`services/document_service.py:136`	At startup, marks documents whose file is gone as `missing`
`FileStorage`	`services/storage.py:7`	Copies PDFs into `STORAGE_DIR`

Function / class	File	What it does
`ingest_document`	`services/ingestion.py:33`	Claims the document, extracts, chunks, embeds in batches, saves; on error sets `failed`
`pending_document_ids`	`services/ingestion.py:152`	At startup, finds unfinished documents to resume
`extract_pdf`	`services/pdf_extraction.py:53`	PyMuPDF / pymupdf4llm text per page, flags pages needing OCR
`chunk_pages`, `chunk_page`	`services/chunking.py:105`, `:73`	Page-bound chunks (1800 chars, 200 overlap), tables kept whole
`retrieve`	`services/retrieval.py:34`	Embeds the query, cosine search in pgvector, filters by scope
`ask`	`services/answer_service.py:41`	Full chat turn: history, retrieval, Groq, citation check, save
`build_messages`, `normalize`	`rag/pipeline.py:44`, `:65`	Builds the prompt; validates `S#` citations and renumbers them `[n]`
Prompts	`rag/prompts/basic.py`, `extensive.py`, `quiz.py`, `common.py`	System prompts per mode

Function / class	File	What it does
`generate_quiz`, `submit_attempt`	`services/quiz_service.py:104`, `:201`	Builds questions from chunks; grades answers
Conversations	`services/conversation_service.py:30` onward	Create, update, history for prompts
App startup	`main.py:22` (`lifespan`), `:44` (`create_app`)	Checks providers, marks missing files, resumes ingestion
Settings	`core/config.py:17`	All `.env` values

Frontend classes

Pages own screen state as signals and call injectable API services; every HTTP call passes through the auth and error interceptors.

```
classDiagram
  class LibraryPage {
    documents: signal
    busy: computed
    refresh()
    delete()
  }
  class UploadDropzone {
    files: output
  }
  class WorkspacePage {
    conversation: signal
    messages: signal
    send()
    openCitation()
  }
  class Composer {
    mode: model
    scope: model
    submit: output
  }
  class ChatMessage {
```

```
    message: input
    cite: output
  }
  class AttachDialog
  class ReaderPage
  class QuizListPage
  class QuizRunnerPage
  class QuizResultPage
  class PdfViewer {
    data: input
    page: model
    highlight(quote)
  }
  class DocumentsService {
    list()
    upload()
    file()
    delete()
  }
  class ConversationsService {
    create()
    get()
    update()
    ask()
  }
  class QuizService {
    create()
    get()
    submit()
  }
  class SettingsService
  class AuthService {
    isAuthenticated()
    token()
  }
  class ToastService
  LibraryPage *-- UploadDropzone
  LibraryPage --> DocumentsService
  LibraryPage --> ConversationsService
  WorkspacePage *-- Composer
  WorkspacePage *-- ChatMessage
  WorkspacePage *-- AttachDialog
  WorkspacePage *-- PdfViewer
```

```
  WorkspacePage --> ConversationsService
  WorkspacePage --> DocumentsService
  ReaderPage *-- PdfViewer
  QuizListPage --> QuizService
  QuizRunnerPage --> QuizService
  QuizResultPage --> QuizService
  LibraryPage --> ToastService
  AuthService <.. authInterceptor
  AuthService <.. authGuard
```

Class	File	Job
`LibraryPage`	`features/library/library-page.ts:24`	PDF list, upload, delete, polling, “Study together” / “Quiz me”
`UploadDropzone`	`features/library/upload-dropzone.ts:76`	Drag-and-drop or browse for PDFs
`WorkspacePage`	`features/workspace/workspace-page.ts:34`	Sessions rail, viewer tabs, chat, rename/attach/delete
`Composer`	`features/workspace/composer.ts:59`	Question box, Basic/Extensive and scope toggles
`ChatMessage`	`features/workspace/chat-message.ts:15`	One answer: clarification, supplementary block, sources
`AttachDialog`	`features/workspace/attach-dialog.ts:61`	Choose which PDFs a chat uses
`ReaderPage`	`features/reader/reader-page.ts:55`	Full-screen viewer at `?page=&quote=`
`QuizListPage`, `QuizRunnerPage`, `QuizResultPage`	`features/quiz/quiz-list-page.ts:20`, `quiz-runner-page.ts:119`, `quiz-result-page.ts:88`	Make, take and review quizzes
`PdfViewer`	`shared/ui/pdf-viewer.ts:46`	pdf.js rendering, zoom, page jump, quote highlight
`RichText`, `Icon`, `ConfirmDialog`, `ToastHost`	`shared/ui/`	Shared UI pieces

Class	File	Job
`DocumentsService`	`core/services/documents.service.ts:8`	`/documents` calls
`ConversationsService`	`core/services/conversations.service.ts:10`	`/conversations` and `/messages` calls
`QuizService`	`core/services/quiz.service.ts:8`	`/quizzes` calls
`SettingsService`, `ThemeService`, `ToastService`	`core/services/`	Default scope, light/dark theme, notifications
`AuthService`, `authInterceptor`, `authGuard`	`core/auth/auth.service.ts:13`, `auth.interceptor.ts:6`, `auth.guard.ts:6`	Auth placeholders; `AUTH_ENABLED = false`
`errorInterceptor`, `apiErrorMessage`	`core/http/error.interceptor.ts:8`, `api-error.ts:4`	Readable error messages
Routes	`app.routes.ts:4`	`/library`, `/study/:id`, `/read/:id`, `/quizzes/...`

Data flow

Uploads return immediately and are processed in the background; chat and quiz calls are synchronous and wait for Groq.

Upload and ingestion

```
sequenceDiagram
  actor S as Student
  participant LP as LibraryPage
  participant DS as DocumentsService
  participant R as routes/documents.py
  participant Svc as document_service
  participant FS as FileStorage
  participant DB as Postgres
  participant Ing as ingestion
  participant Q as Qwen3EmbeddingProvider
  S->>LP: drop PDF
  LP->>DS: upload(file)
  DS->>R: POST /documents
  R->>Svc: upload_document()
```

```
  Svc->>Svc: size + PDF header check, sha256
  alt same file already uploaded
    Svc-->>R: existing document (created=false)
  else new file
    Svc->>FS: save copy
    Svc->>DB: INSERT Document(status=pending)
    R-)Ing: background task ingest_document(id)
  end
  R-->>LP: 201 DocumentOut(pending)
  Ing->>DB: claim: pending to processing
  Ing->>Ing: extract_pdf() then chunk_pages()
  loop batches of 16 chunks
    Ing->>Q: embed_documents(texts)
    Q-->>Ing: 1024-dim vectors
  end
  Ing->>DB: INSERT pages + chunks, status=completed
  Note over Ing,DB: on any error: delete chunks, status=failed, save error
  loop every 2.5 s while any document is pending/processing
    LP->>DS: list()
    DS->>R: GET /documents
    R-->>LP: statuses
  end
```

Where: route `api/routes/documents.py:28`, `upload_document` at `services/document_service.py:37`, `ingest_document` at `services/ingestion.py:33`, embedding at `rag/providers/embeddings/qwen3.py:10`.

Asking a question

```
sequenceDiagram
  actor S as Student
  participant WP as WorkspacePage
  participant CS as ConversationsService
  participant R as routes/chat.py
  participant A as answer_service.ask
  participant Ret as retrieval
  participant Q as Qwen3EmbeddingProvider
  participant DB as Postgres + pgvector
  participant P as pipeline
  participant G as GroqLLMProvider
  S->>WP: type question (mode, scope)
  WP->>CS: ask(id, question, mode, scope)
```

```
  CS->>R: POST /conversations/{id}/messages
  R->>A: ask()
  A->>DB: load recent history (HISTORY_MAX_MESSAGES)
  A->>Ret: retrieve(query, scope)
  Ret->>Q: embed_query(query)
  Ret->>DB: cosine search, top 8, scope filter
  DB-->>Ret: chunks with filename + page
  alt basic mode and no chunks
    A-->>R: insufficient_context (Groq not called)
  else
    A->>P: build_messages(prompt, history, S1..S8)
    A->>G: complete(json_mode)
    G-->>A: JSON answer citing S# ids
    A->>P: normalize(): drop invented ids, renumber [n]
  end
  A->>DB: INSERT Message + MessageSource snapshots
  R-->>WP: AskResponse(answer, sources)
  S->>WP: click [n]
  WP->>WP: PdfViewer opens page, highlights quote
```

Where: route `api/routes/chat.py:16`, `ask` at `services/answer_service.py:41`, `retrieve` at `services/retrieval.py:34`, `normalize` at `rag/pipeline.py:65`.

In basic mode an answer with no valid citation becomes `insufficient_context`; in extensive mode uncited text moves to `supplementary`.

Quizzes

```
sequenceDiagram
  participant QP as QuizListPage
  participant R as routes/quizzes.py
  participant QS as quiz_service
  participant DB as Postgres
  participant G as GroqLLMProvider
  QP->>R: POST /quizzes (document_ids, topic?, count)
  R->>QS: generate_quiz()
  alt topic given
    QS->>DB: retrieve chunks for topic
  else
    QS->>DB: random sample of chunks
  end
  QS->>G: complete(quiz prompt, json_mode)
  G-->>QS: questions with S# sources
```

```
  QS->>QS: drop invalid questions
  QS->>DB: INSERT Quiz + QuizQuestion
  R-->>QP: QuizOut (no answer key)
  QP->>R: POST /quizzes/{id}/attempts (answers)
  R->>QS: submit_attempt()
  QS->>DB: INSERT QuizAttempt(score)
  R-->>QP: score + answer key + explanations
```

Where: routes `api/routes/quizzes.py:57` and `:118`, `generate_quiz` at `services/quiz_service.py:104`, `submit_attempt` at `:201`.

Status updates: polling

Yes, the frontend polls: `LibraryPage` calls `GET /documents` every 2.5 seconds, but only while at least one PDF is `pending` or `processing`, and stops once all are `completed`, `failed` or `missing`.

- Interval: `POLL_MS = 2500` at `features/library/library-page.ts:15`; the `setInterval` effect is at `library-page.ts:58`.
- A toast appears when a document becomes ready. Chat and quizzes do not poll; they wait for the HTTP response.
- Replacing polling later: a Server-Sent Events endpoint fed by `ingest_document`, with `LibraryPage` subscribing instead of its timer.

```
stateDiagram-v2
  [*] --> pending: upload_document()
  pending --> processing: ingest_document() claims it
  processing --> completed: chunks + vectors saved
  processing --> failed: extraction or embedding error
  processing --> pending: backend restarted mid-ingest
  failed --> pending: same PDF uploaded again
  completed --> missing: file gone at startup
  missing --> pending: same PDF uploaded again
  completed --> [*]: delete_document()
```

Statuses are defined in `DocumentStatus` (`models/enums.py:4`); the restart reset is in `pending_document_ids` (`services/ingestion.py:152`), the missing check in `mark_missing_files` (`services/document_service.py:136`).

File index

Area	Path
Backend entry, startup	`Backend/src/backend/main.py`, `__init__.py` (`uv run backend`)
Settings, logging	`Backend/src/backend/core/`
Database session, base classes	`Backend/src/backend/database/`
Tables	`Backend/src/backend/models/`
Request/response shapes	`Backend/src/backend/schemas/`
HTTP endpoints	`Backend/src/backend/api/routes/` (documents, conversations, chat, quizzes, settings)
Local user, storage, ingestor wiring	`Backend/src/backend/api/dependencies.py`
Business logic	`Backend/src/backend/services/`
Prompts, citation checks, providers	`Backend/src/backend/rag/`
Migrations	`Backend/alembic/versions/`
Tests	`Backend/tests/unit/`, `Backend/tests/integration/`, fakes in `Backend/tests/fakes.py`
Postgres container	`Backend/docker-compose.yml`, `Backend/docker/initdb/`
Frontend bootstrap, routes	`Frontend/Noesis/src/app/app.ts`, `app.config.ts`, `app.routes.ts`
API client, auth, errors	`Frontend/Noesis/src/app/core/`
Pages	`Frontend/Noesis/src/app/features/` (library, workspace, reader, quiz)
Shared UI, PDF viewer	`Frontend/Noesis/src/app/shared/`
Design tokens and styles	`Frontend/Noesis/src/styles/`
Dev proxy to backend	`Frontend/Noesis/proxy.conf.json`